Übung 1: Grundlagen iterativer Verfahren

Kurs: Iterative Lösung großer Gleichungssysteme
Semester: Sommersemester 2026
Datum: 23.04.2026
Institut: Institute for Analysis and Scientific Computing, TU Wien

Hosan Youssef: 1648939

Model Problem: 1D-Poisson Equation

We consider the domain $\Omega = (0,1)$ and the **1D-Poisson problem** with homogeneous Dirichlet boundary conditions:

$$\begin{aligned} -u'' &= 1 \quad \text{in } \Omega, \\ u(0) = u(1) &= 0. \end{aligned} \tag{1}$$

Discretization via Finite Differences

Let $n \in \mathbb{N}$, $h = 1/(n+1)$, and the grid points $x_j = jh$ for $j = 0, \dots, n+1$. We approximate the exact solution $u_j \approx u(x_j)$ via finite differences:

$$-\frac{u_{j-1} - 2u_j + u_{j+1}}{h^2} = 1 \quad \text{for } j = 1, \dots, n,$$

with $u_0 = u_{n+1} = 0$. This leads to the linear system $\mathbf{A}\mathbf{u} = \mathbf{b}$ with:

$$\mathbf{A} = \frac{1}{h^2} \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & 2 \end{pmatrix} \in \mathbb{R}^{n \times n} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^n. \tag{2}$$

Existence and Uniqueness: The Lax-Milgram Theorem

To prove the existence of a unique solution for the Poisson problem (1), we consider its **weak formulation**.

1. Weak Formulation

We multiply the equation $-u'' = 1$ by a test function $v \in H_0^1(0,1)$ and integrate over the domain $\Omega = (0,1)$:

$$-\int_0^1 u''(x)v(x)\,dx = \int_0^1 1 \cdot v(x)\,dx$$

Using integration by parts and the boundary conditions $v(0) = v(1) = 0$, we obtain the variational formulation: Find $u \in H_0^1(\Omega)$ such that

$$a(u,v) = F(v) \quad \forall v \in H_0^1(\Omega)$$

where:

- **Bilinear form:** $a(u,v) = \int_0^1 u'(x)v'(x)\,dx$
- **Linear functional:** $F(v) = \int_0^1 v(x)\,dx$

2. Requirements of Lax-Milgram

The Lax-Milgram Theorem guarantees a unique solution if the following conditions are met:

a) Continuity of the Bilinear Form

By the Cauchy-Schwarz inequality:

$$|a(u,v)| = \left|\int_0^1 u'v'\,dx\right| \leq \|u'\|_{L^2}\|v'\|_{L^2} = |u|_{H^1}|v|_{H^1} \leq \|u\|_{H^1}\|v\|_{H^1}$$

Thus, $a(\cdot,\cdot)$ is continuous with constant $M = 1$.

b) Coercivity of the Bilinear Form

Since we are in the space $H_0^1(0,1)$ with zero boundary conditions, the **Poincaré inequality** applies: there exists $C_P > 0$ such that $\|v\|_{L^2} \leq C_P|v|_{H^1}$. It follows that:

$$a(v,v) = \int_0^1 (v')^2\,dx = |v|_{H^1}^2 \geq \alpha\|v\|_{H^1}^2$$

The bilinear form is therefore coercive.

c) Continuity of the Linear Functional

Since the right-hand side $f = 1$ is in $L^2(0,1)$, we have:

$$|F(v)| = \left|\int_0^1 1 \cdot v\,dx\right| \leq \|1\|_{L^2}\|v\|_{L^2} \leq 1 \cdot \|v\|_{H^1}$$

Thus, $F(\cdot)$ is a bounded (continuous) linear functional.

Conclusion

As all requirements are satisfied, the **Lax-Milgram Theorem** ensures the existence and uniqueness of a weak solution $u \in H_0^1(\Omega)$.

Exercise 1 (Damped Richardson)

1. Implement a function `[A,b] = assemble(n)` that assembles the matrix A and right-hand side b.
2. Implement the damped Richardson method: `val = Richardson(A, b, x0, w, k)`.
3. Compare different values of ω, including $\omega = h^2/2$. Consider $n \in \{8, 32, 128\}$ and $k = 500$. Plot the error versus the iteration number in a semilogarithmic plot.

The Damped Richardson method is a stationary iterative method used to solve the linear system $\mathbf{Au} = \mathbf{b}$. It updates the solution by moving in the direction of the residual.

1. Mathematical Formulation

Given an initial guess $\mathbf{x}_0 \in \mathbb{R}^n$, the iteration scheme is defined as:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \omega(\mathbf{b} - \mathbf{A}\mathbf{x}_k) \tag{3}$$

where:

- $\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$ is the **residual** at step k.
- $\omega > 0$ is the **damping parameter** (weight).
- The iteration matrix is given by $\mathbf{M}_{Rich} = \mathbf{I} - \omega\mathbf{A}$.

Convergence

For the method to converge ($\rho(\mathbf{M}_{Rich}) < 1$), the weight must satisfy $0 < \omega < 2/\lambda_{max}(\mathbf{A})$. In this exercise, we specifically test $\omega = h^2/2$.

---

2. Implementation

The following functions assemble the system matrix and implement the iterative solver.

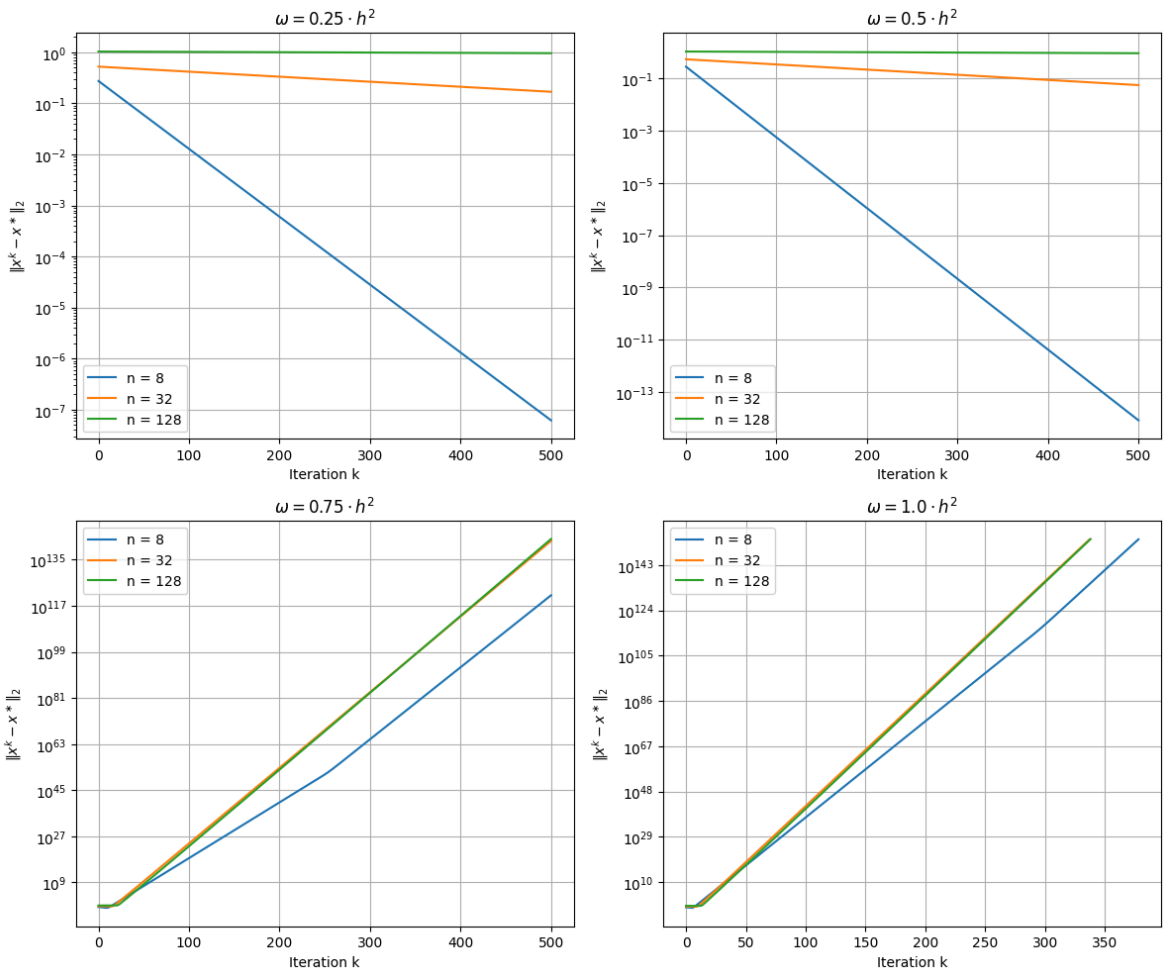
System Assembly

The function `assemble(n)` creates the Poisson matrix $\mathbf{A}$ and the right-hand side $\mathbf{b}$ based on finite differences.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```python
def assemble(n):
    h = 1.0 / (n + 1)
    A = (1.0 / h**2) * (
        2*np.eye(n) - np.eye(n, k=1) - np.eye(n, k=-1)
    )
    b = np.ones(n)
    return A, b

def Richardson(A, b, x0, w, k, x_exact):
    x = x0.copy()
    errors = np.zeros(k + 1)
    errors[0] = np.linalg.norm(x - x_exact)

    for j in range(1, k + 1):
        x = x + w * (b - A @ x)
        errors[j] = np.linalg.norm(x - x_exact)

    return x, errors


ns = [8, 32, 128]
k = 500

# Dummy h für Definition der Omegas (wird später pro n neu gesetzt)
h_dummy = 1.0 / (128 + 1)

omega_factors = [1/4, 1/2, 3/4, 1.0]  # multipliziert mit h^2

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for idx, factor in enumerate(omega_factors):
    ax = axes[idx // 2, idx % 2]

    for n in ns:
        A, b = assemble(n)
        h = 1.0 / (n + 1)
        w = factor * h**2

        x_exact = np.linalg.solve(A, b)
        x0 = np.zeros(n)

        _, errors = Richardson(A, b, x0, w, k, x_exact)

        ax.semilogy(range(k + 1), errors, label=f"n = {n}")

    ax.set_title(rf"$\omega = {factor} \cdot h^2$")
    ax.set_xlabel("Iteration k")
    ax.set_ylabel(r"$\|x^k - x^\ast\|_2$")
    ax.grid(True)
    ax.legend()

plt.tight_layout()
plt.show()
```



2. Analysis of the Richardson Method

Iteration matrix

The Richardson iteration is given by:

$$x^{k+1} = x^k + \omega(b - Ax^k),$$

which leads to the iteration matrix:

$$M = I - \omega A.$$

Eigenvalues of the iteration matrix

If

$$Av_j = \lambda_j v_j,$$

then

$$Mv_j = (1 - \omega\lambda_j)v_j,$$

so the eigenvalues of the iteration matrix are:

$$\mu_j = 1 - \omega\lambda_j.$$

Convergence condition

The Richardson method converges if and only if

$$\rho(I - \omega A) = \max_j |\mu_j| < 1,$$

which is equivalent to

$$|1 - \omega\lambda_j| < 1 \quad \text{for all } j.$$

This yields the condition:

$$0 < \omega < \frac{2}{\lambda_{\max}(A)}.$$

Choice of the relaxation parameter ω

From the eigenvalue estimate

$$\lambda_{\max}(A) \approx \frac{4}{h^2},$$

we obtain the stability condition:

$$\omega < \frac{h^2}{2}.$$

We therefore choose

$$\omega = \frac{h^2}{2},$$

which satisfies the convergence condition.

> **Remark:**
> In Exercise 6, we will show that this choice is in fact the *optimal* damping parameter.

Special case: $\omega = \frac{h^2}{2}$

With this choice,

$$A = \frac{1}{h^2}T \quad \Rightarrow \quad M = I - \omega A = I - \frac{1}{2}T.$$

The eigenvalues become:

$$\mu_j = 1 - 2\sin^2\left(\frac{j\pi}{2(n+1)}\right) = \cos\left(\frac{j\pi}{n+1}\right).$$

Spectral radius

Thus,

$$\rho(I - \omega A) = \max_j |\mu_j| = \cos\left(\frac{\pi}{n+1}\right).$$

Observations for different n

- **For $n = 8$:**

$$\rho \approx \cos(\pi/9) \approx 0.94$$

- **For $n = 32$:**

$$\rho \approx \cos(\pi/33) \approx 0.995$$

- **For $n = 128$:**

$$\rho \approx \cos(\pi/129) \approx 0.9997$$

Conclusion

As $n \to \infty$, we have

$$\rho(I - \omega A) \to 1.$$

Thus, the convergence of the Richardson method becomes increasingly slow for finer grids.

👉 This explains why the error curves flatten for large n, making the method inefficient in practice.

Exercise 2 (Jacobi and Gauss-Seidel)

1. Implement the Jacobi and Gauss-Seidel methods: `val = Jacobi(A, b, x0, k)` and `val = GaussSeidel(A, b, x0, k)`.
2. Compare all three methods (Richardson, Jacobi, GS) for $n \in \{8, 32, 128\}$ and $k = 500$. Plot the errors.

Jacobi Method

1. Mathematical Formulation (Splitting Approach)

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be regular and define
$\mathbf{D} \in \mathbb{R}^{n \times n}$ by

$$D_{ij} = A_{ij}\delta_{ij},$$

i.e., $\mathbf{D}$ is the diagonal of $\mathbf{A}$.

Assume that $\mathbf{D}$ is invertible, i.e., $A_{ii} \neq 0$ for all $i = 1, \ldots, n$.

Splitting

We write

$$\mathbf{A} = \mathbf{D} + (\mathbf{A} - \mathbf{D}).$$

Define

$$\mathbf{G} := \mathbf{D}, \qquad \mathbf{H} := -(\mathbf{A} - \mathbf{D}).$$

Then

$$\mathbf{A} = \mathbf{G} - \mathbf{H}.$$

Iterative Scheme

The Jacobi iteration is defined via:

- Solve

$$\mathbf{D}\,\delta_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$$

- Update

$$\mathbf{x}_{k+1} := \mathbf{x}_k + \delta_k$$

Explicit Form

Substituting δ_k gives

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{D}^{-1}(\mathbf{b} - \mathbf{A}\mathbf{x}_k).$$

Using $\mathbf{A} = \mathbf{D} + (\mathbf{A} - \mathbf{D})$, we obtain

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{D}^{-1}\big(\mathbf{b} - \mathbf{D}\mathbf{x}_k - (\mathbf{A} - \mathbf{D})\mathbf{x}_k\big).$$

Rearranging yields

$$\mathbf{x}_{k+1} = \mathbf{D}^{-1}\left(\mathbf{b} - (\mathbf{A} - \mathbf{D})\mathbf{x}_k\right).$$

Component-wise Form

Using upper indices for the iteration and lower indices for components, we obtain

$$x_i^{k+1} = x_i^k + \frac{1}{A_{ii}}\left(b_i - \sum_{j=1}^{n} A_{ij} x_j^k\right), \qquad i = 1, \ldots, n.$$

Equivalently,

$$x_i^{k+1} = \frac{1}{A_{ii}} \left(b_i - \sum_{j \neq i} A_{ij} x_j^k \right).$$

Interpretation

- Each component is computed **independently**.
- Only values from the previous iterate $\mathbf{x}_k$ are used.
- No newly computed values are reused within the same iteration.
- This makes the Jacobi method naturally **parallelizable**, but leads to slower information propagation compared to Gauss–Seidel.

Convergence

The method converges if

$$\rho(\mathbf{M}_{Jac}) < 1.$$

For the discrete 1D Poisson problem:

- The Jacobi method is convergent.
- However, it converges more slowly than Gauss–Seidel.
- Moreover,

  $$\rho(\mathbf{M}_{Jac}) \to 1 \quad \text{as } n \to \infty,$$

  which implies very slow convergence for large systems.

Gauss–Seidel Method

1. Mathematical Formulation (Splitting Approach)

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be regular and write

$$\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{U},$$

where:

- $\mathbf{D}$ is the diagonal of $\mathbf{A}$,
- $\mathbf{L}$ is the strictly lower triangular part,
- $\mathbf{U}$ is the strictly upper triangular part.

Assume that $\mathbf{D}$ (and hence $\mathbf{D} + \mathbf{L}$) is invertible.

Splitting

We consider the splitting

$$\mathbf{A} = (\mathbf{D} + \mathbf{L}) + \mathbf{U}.$$

Define

$$\mathbf{G} := \mathbf{D} + \mathbf{L}, \qquad \mathbf{H} := -\mathbf{U}.$$

Then

$$\mathbf{A} = \mathbf{G} - \mathbf{H}.$$

Iterative Scheme

The Gauss–Seidel iteration is defined via:

- Solve

$$(\mathbf{D} + \mathbf{L})\, \delta_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$$

- Update

$$\mathbf{x}_{k+1} := \mathbf{x}_k + \delta_k$$

Explicit Form

Substituting δ_k gives

$$\mathbf{x}_{k+1} = \mathbf{x}_k + (\mathbf{D} + \mathbf{L})^{-1}(\mathbf{b} - \mathbf{A}\mathbf{x}_k).$$

Using $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$, we obtain

$$\mathbf{x}_{k+1} = \mathbf{x}_k + (\mathbf{D} + \mathbf{L})^{-1}\big(\mathbf{b} - (\mathbf{D} + \mathbf{L} + \mathbf{U})\mathbf{x}_k\big).$$

Rearranging step by step:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + (\mathbf{D} + \mathbf{L})^{-1}\big(\mathbf{b} - (\mathbf{D} + \mathbf{L})\mathbf{x}_k - \mathbf{U}\mathbf{x}_k\big)$$

$$= \mathbf{x}_k + (\mathbf{D} + \mathbf{L})^{-1}(\mathbf{b} - (\mathbf{D} + \mathbf{L})\mathbf{x}_k) - (\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}\mathbf{x}_k$$

Using

$$\mathbf{x}_k + (\mathbf{D} + \mathbf{L})^{-1}(\mathbf{b} - (\mathbf{D} + \mathbf{L})\mathbf{x}_k) = (\mathbf{D} + \mathbf{L})^{-1}\mathbf{b},$$

we finally obtain

$$\mathbf{x}_{k+1} = (\mathbf{D} + \mathbf{L})^{-1}\left(\mathbf{b} - \mathbf{U}\mathbf{x}_k\right).$$

Component-wise Form

Using upper indices for iterations and lower indices for components, we obtain

$$x_i^{k+1} = x_i^k + \frac{1}{A_{ii}}\left(b_i - \sum_{j<i} A_{ij}x_j^{k+1} - \sum_{j\geq i} A_{ij}x_j^k\right), \qquad i = 1, \ldots, n.$$

Equivalently,

$$x_i^{k+1} = \frac{1}{A_{ii}} \left(b_i - \sum_{j<i} A_{ij} x_j^{k+1} - \sum_{j>i} A_{ij} x_j^k \right), \qquad i = 1, \dots, n.$$

Interpretation

- The components are updated **sequentially**.
- Newly computed values x_j^{k+1} are used immediately for $j < i$.
- This leads to faster propagation of information compared to Jacobi.

Convergence

The method converges if

$$\rho(\mathbf{M}_{GS}) < 1.$$

For the discrete 1D Poisson problem:

- Gauss–Seidel is convergent.
- It typically converges faster than Jacobi.
- However,

  $$\rho(\mathbf{M}_{GS}) \to 1 \quad \text{as } n \to \infty,$$

  so convergence deteriorates for large systems.

In [3]:
```
import numpy as np

def Jacobi(A, b, x0, k, x_exact=None):
    D = np.diag(A)                  # diagonal entries of A
    R = A - np.diagflat(D)          # strictly off-diagonal part = A - D

    x = x0.copy()
    errors = np.zeros(k + 1)

    if x_exact is not None:
        errors[0] = np.linalg.norm(x - x_exact)

    for m in range(1, k + 1):
        x = (b - R @ x) / D

        if x_exact is not None:
            errors[m] = np.linalg.norm(x - x_exact)

    return x, errors
```

The term `R @ x` corresponds to the sum

$$\sum_{j \neq i} A_{ij} x_j,$$

since $R = A - \mathrm{diag}(A)$ contains only the off-diagonal entries of A.

In [4]:
```
import numpy as np

def GaussSeidel(A, b, x0, k, x_exact=None):
    n = len(b)
    x = x0.copy()
    errors = np.zeros(k + 1)

    if x_exact is not None:
        errors[0] = np.linalg.norm(x - x_exact)

    for m in range(1, k + 1):
        x_old = x.copy()

        for i in range(n):
            sum1 = 0.0
            for j in range(i):
                sum1 += A[i, j] * x[j]          # new values x_j^(k+1), j < i

            sum2 = 0.0
            for j in range(i + 1, n):
                sum2 += A[i, j] * x_old[j]      # old values x_j^k, j > i

            x[i] = (b[i] - sum1 - sum2) / A[i, i]

        if x_exact is not None:
            errors[m] = np.linalg.norm(x - x_exact)

    return x, errors
```

In the Gauss–Seidel method, the update is performed sequentially.

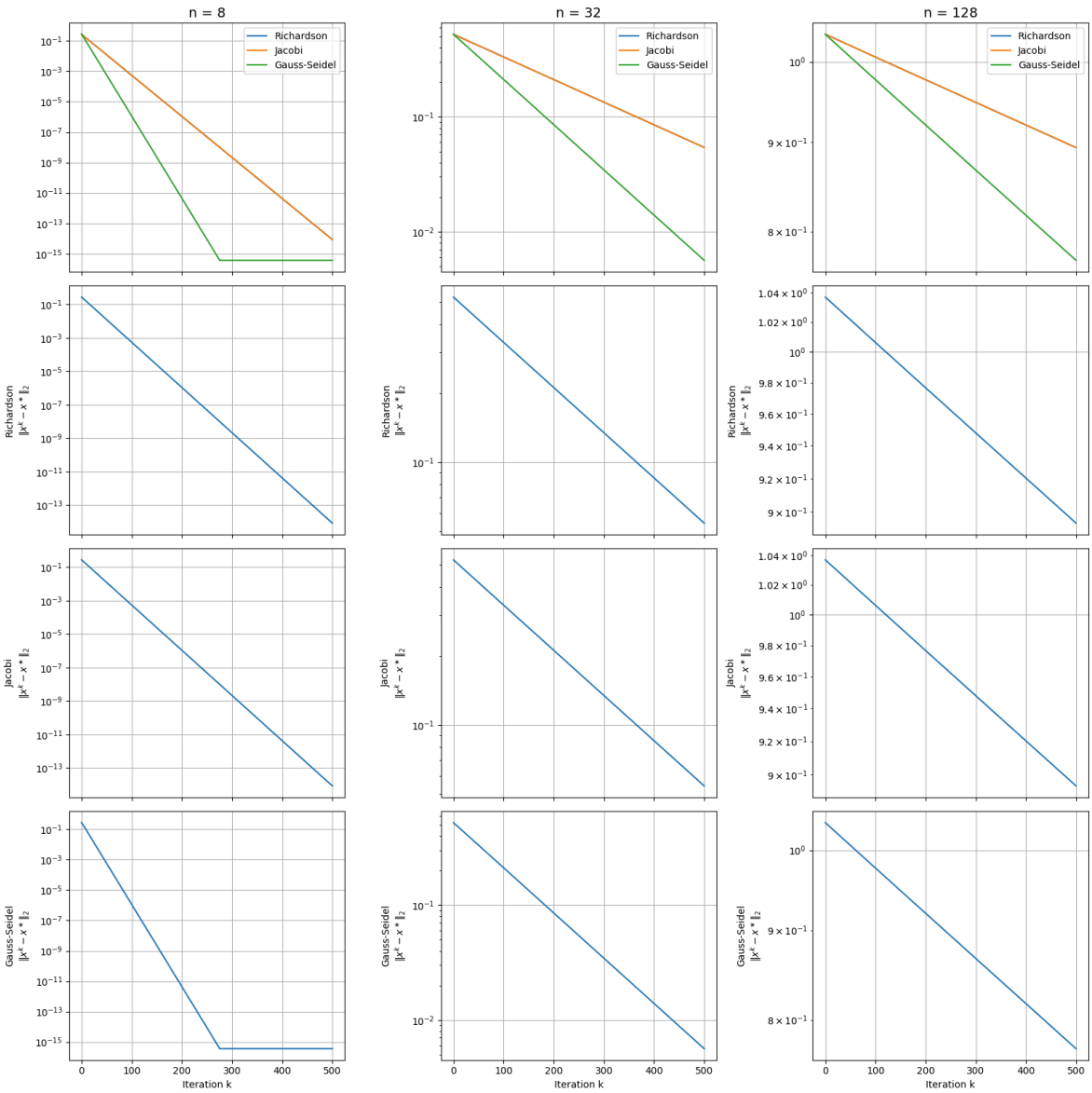
For each component x_i^{k+1}:

- the values x_j^{k+1} for $j < i$ are already updated and reused,
- the values x_j^k for $j > i$ are still from the previous iteration.

This is why the implementation requires a loop and cannot be written as a simple matrix-vector product.

In [5]:
```
import numpy as np
import matplotlib.pyplot as plt

ns = [8, 32, 128]
k = 500

fig, axes = plt.subplots(4, 3, figsize=(16, 16), sharex=True)

method_names = ["Richardson", "Jacobi", "Gauss-Seidel"]

for col, n in enumerate(ns):
    A, b = assemble(n)
    h = 1.0 / (n + 1)
    w = h**2 / 2

    x_exact = np.linalg.solve(A, b)
    x0 = np.zeros(n)
```

```
    _, err_rich = Richardson(A, b, x0, w, k, x_exact)
    _, err_jac  = Jacobi(A, b, x0, k, x_exact)
    _, err_gs   = GaussSeidel(A, b, x0, k, x_exact)

    # --- Row 0: ALL METHODS TOGETHER ---
    ax = axes[0, col]
    ax.semilogy(range(k + 1), err_rich, label="Richardson")
    ax.semilogy(range(k + 1), err_jac,  label="Jacobi")
    ax.semilogy(range(k + 1), err_gs,   label="Gauss-Seidel")

    ax.set_title(f"n = {n}", fontsize=14)
    ax.grid(True)
    ax.legend()

    # --- Row 1: Richardson ---
    ax = axes[1, col]
    ax.semilogy(range(k + 1), err_rich)
    ax.set_ylabel("Richardson\n" + r"$\|x^k - x^\ast\|_2$")
    ax.grid(True)

    # --- Row 2: Jacobi ---
    ax = axes[2, col]
    ax.semilogy(range(k + 1), err_jac)
    ax.set_ylabel("Jacobi\n" + r"$\|x^k - x^\ast\|_2$")
    ax.grid(True)

    # --- Row 3: Gauss-Seidel ---
    ax = axes[3, col]
    ax.semilogy(range(k + 1), err_gs)
    ax.set_ylabel("Gauss-Seidel\n" + r"$\|x^k - x^\ast\|_2$")
    ax.set_xlabel("Iteration k")
    ax.grid(True)

plt.tight_layout()
plt.show()
```



Convergence and Comparison of Jacobi and Gauss–Seidel

Lemma (Diagonal dominance and irreducibility)

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be diagonally dominant and irreducible. Then the Jacobi and Gauss–Seidel iterations are well-defined and convergent.

Application to the model problem

For the discrete 1D Poisson problem, the matrix $\mathbf{A}$ has the form

$$A = \frac{1}{h^2} \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ & & -1 & 2 \end{pmatrix}.$$

- For all rows:

$$|A_{ii}| = \frac{2}{h^2}, \qquad \sum_{j \neq i} |A_{ij}| \leq \frac{2}{h^2}.$$

- For the first and last rows:

$$|A_{ii}| = \frac{2}{h^2} > \frac{1}{h^2} = \sum_{j \neq i} |A_{ij}|.$$

Hence, $\mathbf{A}$ is diagonally dominant, and strictly diagonally dominant in at least one row.

Moreover, the matrix is irreducible due to its tridiagonal structure.

Therefore, the lemma applies, and both Jacobi and Gauss–Seidel methods converge.

Stein–Rosenberg Theorem

Let $\mathbf{A} = (a_{ij})$ with

$$a_{ii} > 0, \qquad a_{ij} \leq 0 \quad \text{for } i \neq j.$$

Let $\mathbf{T}_J$ and $\mathbf{T}_{GS}$ denote the iteration matrices of Jacobi and Gauss–Seidel.

Then exactly one of the following holds:

1. $\rho(\mathbf{T}_J) = \rho(\mathbf{T}_{GS}) = 0$
2. $0 < \rho(\mathbf{T}_{GS}) < \rho(\mathbf{T}_J) < 1$
3. $\rho(\mathbf{T}_J) = \rho(\mathbf{T}_{GS}) = 1$
4. $1 < \rho(\mathbf{T}_J) < \rho(\mathbf{T}_{GS})$

Application of Stein–Rosenberg

The matrix $\mathbf{A}$ satisfies:

- $a_{ii} > 0$
- $a_{ij} \leq 0$ for $i \neq j$

Thus, the Stein–Rosenberg theorem applies.

From the previous lemma, both methods are convergent, hence

$$\rho(\mathbf{T}_J) < 1.$$

Therefore, only case (2) is possible:

$$0 < \rho(\mathbf{T}_{GS}) < \rho(\mathbf{T}_J) < 1.$$

Conclusion

Gauss–Seidel has a strictly smaller spectral radius than Jacobi and therefore converges faster.

This theoretical result is consistent with the observed numerical behavior.

Relation between Richardson and Jacobi

The Richardson iteration is given by

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \omega\big(\mathbf{b} - \mathbf{A}\mathbf{x}^k\big).$$

For the model problem, the matrix has the form

$$\mathbf{A} = \frac{1}{h^2}\mathbf{T},$$

where $\mathbf{T}$ is the tridiagonal matrix with diagonal entries 2 and off-diagonal entries -1.

We now choose

$$\omega = \frac{h^2}{2}.$$

Then the Richardson iteration becomes

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \frac{h^2}{2}\left(\mathbf{b} - \frac{1}{h^2}\mathbf{T}\mathbf{x}^k\right) = \mathbf{x}^k + \frac{h^2}{2}\mathbf{b} - \frac{1}{2}\mathbf{T}\mathbf{x}^k.$$

Using

$$\mathbf{T} = 2\mathbf{I} + \mathbf{L} + \mathbf{U},$$

we obtain

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \frac{h^2}{2}\mathbf{b} - \frac{1}{2}(2\mathbf{I} + \mathbf{L} + \mathbf{U})\mathbf{x}^k.$$

Hence

$$\mathbf{x}^{k+1} = \frac{h^2}{2}\mathbf{b} - \frac{1}{2}(\mathbf{L} + \mathbf{U})\mathbf{x}^k.$$

Jacobi method

For the Jacobi method, we split

$$\mathbf{A} = \mathbf{D} + (\mathbf{A} - \mathbf{D}),$$

where

$$\mathbf{D} = \frac{2}{h^2}\mathbf{I}.$$

Thus,

$$\mathbf{D}^{-1} = \frac{h^2}{2}\mathbf{I}.$$

The Jacobi iteration is

$$\mathbf{x}^{k+1} = \mathbf{D}^{-1}\big(\mathbf{b} - (\mathbf{A} - \mathbf{D})\mathbf{x}^k\big).$$

Since

$$\mathbf{A} - \mathbf{D} = \frac{1}{h^2}(\mathbf{L} + \mathbf{U}),$$

it follows that

$$\mathbf{x}^{k+1} = \frac{h^2}{2}\left(\mathbf{b} - \frac{1}{h^2}(\mathbf{L} + \mathbf{U})\mathbf{x}^k\right) = \frac{h^2}{2}\mathbf{b} - \frac{1}{2}(\mathbf{L} + \mathbf{U})\mathbf{x}^k.$$

Conclusion

Thus, for the choice

$$\omega = \frac{h^2}{2},$$

the Richardson iteration and the Jacobi iteration are identical.

In particular, they have the same iteration matrix and therefore exactly the same convergence behavior for the model problem.

Exercise 3 (Stopping Criterion)

Instead of a fixed number of steps k, use a tolerance $\tau > 0$ such that $\|x - x_k\|_2 < \tau$.

1. Derive a computable **a-posteriori error estimator** η_k such that $\|x - x_k\| \leq C\eta_k$.
2. Modify your implementations and test for $\tau \in \{10^{-3}, 10^{-5}, 10^{-7}\}$.

A-posteriori error estimate via Banach's fixed point theorem

Banach fixed point theorem

Let $(X, \|\cdot\|)$ be a Banach space and let $\Phi : X \to X$ satisfy

$$\|\Phi(x) - \Phi(y)\| \leq q\|x - y\| \quad \forall x, y \in X,$$

for some constant $0 < q < 1$.

Then there exists a unique fixed point $x^* \in X$ such that

$$\Phi(x^*) = x^*,$$

and the iterates defined by

$$x_{k+1} := \Phi(x_k)$$

converge to x^*.

Moreover, the following estimates hold:

- **Contraction of the error:**

$$\|x^* - x_{k+1}\| \leq q\|x^* - x_k\|$$

- **A-posteriori error estimate:**

$$\|x^* - x_{k+1}\| \leq \frac{q}{1-q}\|x_{k+1} - x_k\|$$

Application to iterative methods

For linear iterative methods of the form

$$x_{k+1} = Mx_k + Nb,$$

we define the iteration mapping

$$\Phi(x) = Mx + Nb.$$

Then

$$\Phi(x) - \Phi(y) = M(x - y),$$

and hence

$$\|\Phi(x) - \Phi(y)\| \leq \|M\|\|x - y\|.$$

Thus, if $\|M\| < 1$, the mapping Φ is a contraction with

$$q = \|M\|.$$

Since the iteration matrix M satisfies $\rho(M) < 1$, there exists a matrix norm such that $\|M\| < 1$. Hence, the iteration is a contraction, and Banach's fixed point theorem applies.

Error estimator

Let $x^* = x$ denote the exact solution of the system.

From the a-posteriori estimate, we obtain

$$\|x - x_{k+1}\| \leq \frac{q}{1-q}\|x_{k+1} - x_k\|.$$

We therefore define the computable error estimator

$$\eta_k := \|x_k - x_{k-1}\|.$$

Hence,

$$\|x - x_k\| \le C\eta_k, \qquad C = \frac{q}{1-q}.$$

Stopping criterion

We stop the iteration as soon as

$$\eta_k < \tau,$$

which ensures that the true error is controlled up to the constant C.

Strictly speaking, the stopping criterion ensures

$$\|x - x_k\| \le C\tau,$$

i.e., the true error is controlled up to the constant C.

In [6]:
```
def assemble(n):
    h = 1.0 / (n + 1)
    A = (1.0 / h**2) * (2*np.eye(n) - np.eye(n, k=1) - np.eye(n, k=-1))
    b = np.ones(n)
    return A, b
```

In [7]:
```
def Richardson_stop(A, b, x0, w, tau, maxit=100000):
    x_old = x0.copy()
    x_new = x_old + w * (b - A @ x_old)
    k = 1

    while np.linalg.norm(x_new - x_old) >= tau and k < maxit:
        x_old = x_new.copy()
        x_new = x_old + w * (b - A @ x_old)
        k += 1

    return x_new, k
```

In [8]:
```
def Jacobi_stop(A, b, x0, tau, maxit=100000):
    D = np.diag(A)
    R = A - np.diagflat(D)

    x_old = x0.copy()
    x_new = (b - R @ x_old) / D
    k = 1

    while np.linalg.norm(x_new - x_old) >= tau and k < maxit:
        x_old = x_new.copy()
        x_new = (b - R @ x_old) / D
        k += 1

    return x_new, k
```

In [9]:
```
def GaussSeidel_stop(A, b, x0, tau, maxit=100000):
    n = len(b)
    x_old = x0.copy()
    x_new = x0.copy()
    k = 0

    while k < maxit:
```

```
        x_prev = x_new.copy()

        for i in range(n):
            sum1 = np.dot(A[i, :i], x_new[:i])
            sum2 = np.dot(A[i, i+1:], x_prev[i+1:])
            x_new[i] = (b[i] - sum1 - sum2) / A[i, i]

        k += 1

        if np.linalg.norm(x_new - x_prev) < tau:
            break

    return x_new, k
```

In [10]:

```
ns = [8, 32, 128]
taus = [1e-3, 1e-5, 1e-7]

for n in ns:
    A, b = assemble(n)
    h = 1.0 / (n + 1)
    w = h**2 / 2
    x0 = np.zeros(n)
    x_exact = np.linalg.solve(A, b)

    print(f"\n===== n = {n} =====")

    for tau in taus:
        x_rich, k_rich = Richardson_stop(A, b, x0, w, tau)
        x_jac,  k_jac  = Jacobi_stop(A, b, x0, tau)
        x_gs,   k_gs   = GaussSeidel_stop(A, b, x0, tau)

        err_rich = np.linalg.norm(x_exact - x_rich)
        err_jac  = np.linalg.norm(x_exact - x_jac)
        err_gs   = np.linalg.norm(x_exact - x_gs)

        print(f"\ntau = {tau:.0e}")
        print(f"Richardson : iterations = {k_rich:6d}, final exact error = {err_
        print(f"Jacobi     : iterations = {k_jac:6d}, final exact error = {err_j
        print(f"Gauss-Seidel: iterations = {k_gs:6d}, final exact error = {err_g
```

```
===== n = 8 =====

tau = 1e-03
Richardson : iterations =     47, final exact error = 1.470696e-02
Jacobi     : iterations =     47, final exact error = 1.470696e-02
Gauss-Seidel: iterations =     30, final exact error = 6.872979e-03

tau = 1e-05
Richardson : iterations =    121, final exact error = 1.473918e-04
Jacobi     : iterations =    121, final exact error = 1.473918e-04
Gauss-Seidel: iterations =     67, final exact error = 6.888040e-05

tau = 1e-07
Richardson : iterations =    195, final exact error = 1.477147e-06
Jacobi     : iterations =    195, final exact error = 1.477147e-06
Gauss-Seidel: iterations =    104, final exact error = 6.903130e-07

===== n = 32 =====

tau = 1e-03
Richardson : iterations =    192, final exact error = 2.192417e-01
Jacobi     : iterations =    192, final exact error = 2.192417e-01
Gauss-Seidel: iterations =    173, final exact error = 1.093715e-01

tau = 1e-05
Richardson : iterations =   1207, final exact error = 2.189644e-03
Jacobi     : iterations =   1207, final exact error = 2.189644e-03
Gauss-Seidel: iterations =    681, final exact error = 1.087435e-03

tau = 1e-07
Richardson : iterations =   2221, final exact error = 2.196822e-05
Jacobi     : iterations =   2221, final exact error = 2.196822e-05
Gauss-Seidel: iterations =   1188, final exact error = 1.091000e-05

===== n = 128 =====

tau = 1e-03
Richardson : iterations =      1, final exact error = 1.036511e+00
Jacobi     : iterations =      1, final exact error = 1.036511e+00
Gauss-Seidel: iterations =      1, final exact error = 1.036200e+00

tau = 1e-05
Richardson : iterations =  11550, final exact error = 3.370888e-02
Jacobi     : iterations =  11550, final exact error = 3.370888e-02
Gauss-Seidel: iterations =   6945, final exact error = 1.684424e-02

tau = 1e-07
Richardson : iterations =  27078, final exact error = 3.370778e-04
Jacobi     : iterations =  27078, final exact error = 3.370778e-04
Gauss-Seidel: iterations =  14708, final exact error = 1.685368e-04
```

In [11]:

```python
import pandas as pd

results = []

ns = [8, 32, 128]
taus = [1e-3, 1e-5, 1e-7]

for n in ns:
    A, b = assemble(n)
```

```
    h = 1.0 / (n + 1)
    w = h**2 / 2
    x0 = np.zeros(n)
    x_exact = np.linalg.solve(A, b)

    for tau in taus:
        x_rich, k_rich = Richardson_stop(A, b, x0, w, tau)
        x_jac,  k_jac  = Jacobi_stop(A, b, x0, tau)
        x_gs,   k_gs   = GaussSeidel_stop(A, b, x0, tau)

        results.append({
            "Method": "Richardson",
            "n": n,
            "tau": tau,
            "Iterations": k_rich,
            "Final Error": np.linalg.norm(x_exact - x_rich)
        })

        results.append({
            "Method": "Jacobi",
            "n": n,
            "tau": tau,
            "Iterations": k_jac,
            "Final Error": np.linalg.norm(x_exact - x_jac)
        })

        results.append({
            "Method": "Gauss-Seidel",
            "n": n,
            "tau": tau,
            "Iterations": k_gs,
            "Final Error": np.linalg.norm(x_exact - x_gs)
        })

df = pd.DataFrame(results)

df
```

Out[11]:

	Method	n	tau	Iterations	Final Error
0	Richardson	8	1.000000e-03	47	1.470696e-02
1	Jacobi	8	1.000000e-03	47	1.470696e-02
2	Gauss-Seidel	8	1.000000e-03	30	6.872979e-03
3	Richardson	8	1.000000e-05	121	1.473918e-04
4	Jacobi	8	1.000000e-05	121	1.473918e-04
5	Gauss-Seidel	8	1.000000e-05	67	6.888040e-05
6	Richardson	8	1.000000e-07	195	1.477147e-06
7	Jacobi	8	1.000000e-07	195	1.477147e-06
8	Gauss-Seidel	8	1.000000e-07	104	6.903130e-07
9	Richardson	32	1.000000e-03	192	2.192417e-01
10	Jacobi	32	1.000000e-03	192	2.192417e-01
11	Gauss-Seidel	32	1.000000e-03	173	1.093715e-01
12	Richardson	32	1.000000e-05	1207	2.189644e-03
13	Jacobi	32	1.000000e-05	1207	2.189644e-03
14	Gauss-Seidel	32	1.000000e-05	681	1.087435e-03
15	Richardson	32	1.000000e-07	2221	2.196822e-05
16	Jacobi	32	1.000000e-07	2221	2.196822e-05
17	Gauss-Seidel	32	1.000000e-07	1188	1.091000e-05
18	Richardson	128	1.000000e-03	1	1.036511e+00
19	Jacobi	128	1.000000e-03	1	1.036511e+00
20	Gauss-Seidel	128	1.000000e-03	1	1.036200e+00
21	Richardson	128	1.000000e-05	11550	3.370888e-02
22	Jacobi	128	1.000000e-05	11550	3.370888e-02
23	Gauss-Seidel	128	1.000000e-05	6945	1.684424e-02
24	Richardson	128	1.000000e-07	27078	3.370778e-04
25	Jacobi	128	1.000000e-07	27078	3.370778e-04
26	Gauss-Seidel	128	1.000000e-07	14708	1.685368e-04

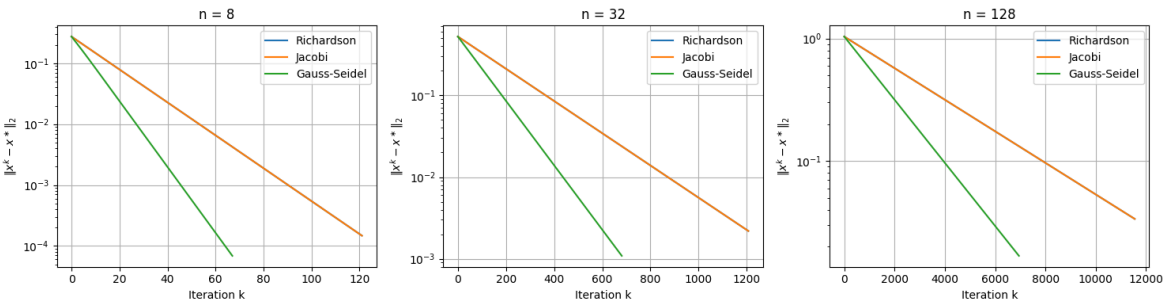
In [12]:

```
import matplotlib.pyplot as plt

def run_with_history(method, A, b, x0, tau, w=None, maxit=100000):
    x = x0.copy()
    errors = []
    steps = []

    x_exact = np.linalg.solve(A, b)
```

```
    for k in range(maxit):
        err = np.linalg.norm(x - x_exact)
        errors.append(err)
        steps.append(k)

        if k > 0 and np.linalg.norm(x - x_prev) < tau:
            break

        x_prev = x.copy()

        if method == "richardson":
            x = x + w * (b - A @ x)

        elif method == "jacobi":
            D = np.diag(A)
            R = A - np.diagflat(D)
            x = (b - R @ x) / D

        elif method == "gs":
            n = len(b)
            x_new = x.copy()
            for i in range(n):
                sum1 = np.dot(A[i, :i], x_new[:i])
                sum2 = np.dot(A[i, i+1:], x[i+1:])
                x_new[i] = (b[i] - sum1 - sum2) / A[i, i]
            x = x_new

    return steps, errors
```

In [13]:

```
ns = [8, 32, 128]
tau = 1e-5

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for i, n in enumerate(ns):
    A, b = assemble(n)
    h = 1.0 / (n + 1)
    w = h**2 / 2
    x0 = np.zeros(n)

    ax = axes[i]

    steps_r, err_r = run_with_history("richardson", A, b, x0, tau, w=w)
    steps_j, err_j = run_with_history("jacobi", A, b, x0, tau)
    steps_g, err_g = run_with_history("gs", A, b, x0, tau)

    ax.semilogy(steps_r, err_r, label="Richardson")
    ax.semilogy(steps_j, err_j, label="Jacobi")
    ax.semilogy(steps_g, err_g, label="Gauss-Seidel")

    ax.set_title(f"n = {n}")
    ax.set_xlabel("Iteration k")
    ax.set_ylabel(r"$\|x^k - x^\ast\|_2$")
    ax.grid(True)
    ax.legend()

plt.tight_layout()
plt.show()
```



Observation

The error decreases approximately linearly on the logarithmic scale for all methods, which indicates linear convergence.

For all problem sizes, the Richardson method with $\omega = h^2/2$ and the Jacobi method show identical behavior. This confirms that both methods coincide for this choice of ω.

The Gauss–Seidel method converges significantly faster than both Richardson and Jacobi. This is consistent with the theoretical result that the spectral radius of the Gauss–Seidel iteration matrix is smaller than that of Jacobi.

Moreover, the convergence becomes slower as the problem size n increases. In particular, for $n = 128$, all methods converge very slowly. This reflects the fact that the spectral radius approaches 1 for finer discretizations.

Consequently, the number of iterations required to reach a prescribed tolerance increases significantly with n, showing that these basic iterative methods are inefficient for large systems.

Exercise 4 (Contraction Factor)

For $n \in \{8, 32, 128\}$ and $k = 150$, compute and plot the experimental contraction factors:

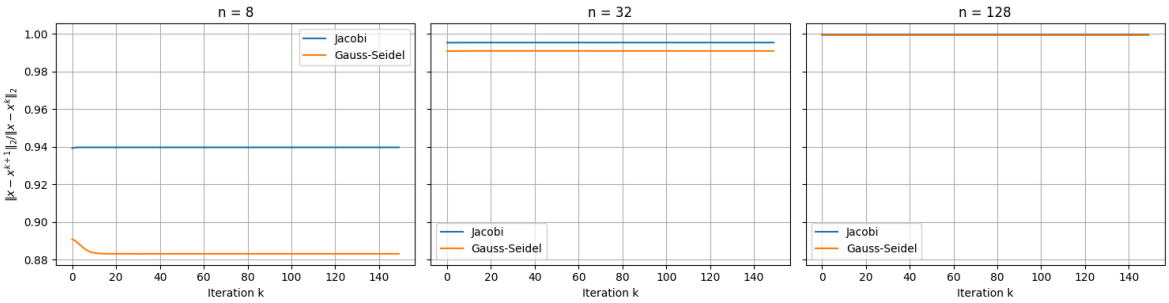
$$\frac{\|x - x_{k+1}\|_2}{\|x - x_k\|_2}$$

What do you observe for Jacobi vs. Gauss-Seidel?

In [14]:
```
def contraction_factors(errors):
    return errors[1:] / errors[:-1]
```

In [15]:
```
ns = [8, 32, 128]
k = 150

fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)

for idx, n in enumerate(ns):
    A, b = assemble(n)
    x0 = np.zeros(n)
    x_exact = np.linalg.solve(A, b)
```

```
    _, err_jac = Jacobi(A, b, x0, k, x_exact=x_exact)
    _, err_gs  = GaussSeidel(A, b, x0, k, x_exact=x_exact)

    q_jac = contraction_factors(err_jac)
    q_gs  = contraction_factors(err_gs)

    ax = axes[idx]
    ax.plot(range(k), q_jac, label="Jacobi")
    ax.plot(range(k), q_gs, label="Gauss-Seidel")
    ax.set_title(f"n = {n}")
    ax.set_xlabel("Iteration k")
    ax.grid(True)
    ax.legend()

axes[0].set_ylabel(r"$\|x-x^{k+1}\|_2 / \|x-x^k\|_2$")
plt.tight_layout()
plt.show()
```



In [16]:
```
for n in ns:
    A, b = assemble(n)
    x0 = np.zeros(n)
    x_exact = np.linalg.solve(A, b)

    _, err_jac = Jacobi(A, b, x0, k, x_exact=x_exact)
    _, err_gs  = GaussSeidel(A, b, x0, k, x_exact=x_exact)

    q_jac = contraction_factors(err_jac)
    q_gs  = contraction_factors(err_gs)

    print(f"n = {n}")
    print(f"  Jacobi last factor       = {q_jac[-1]:.6f}")
    print(f"  Gauss-Seidel last factor = {q_gs[-1]:.6f}")
```

```
n = 8
  Jacobi last factor       = 0.939693
  Gauss-Seidel last factor = 0.883022
n = 32
  Jacobi last factor       = 0.995472
  Gauss-Seidel last factor = 0.990967
n = 128
  Jacobi last factor       = 0.999702
  Gauss-Seidel last factor = 0.999406
```

Observation

The experimental contraction factors

$$\frac{\|x - x^{k+1}\|_2}{\|x - x^k\|_2}$$

approach a constant value as the iteration proceeds. This indicates that, asymptotically, the error behaves like

$$\|e^{k+1}\| \approx \rho(M)\,\|e^k\|,$$

i.e., the convergence rate is governed by the spectral radius of the iteration matrix.

For the Jacobi method, the observed contraction factors agree with the theoretical value

$$\rho_J = \cos\left(\frac{\pi}{n+1}\right),$$

which was derived earlier for the Richardson method with $\omega = h^2/2$. Since Jacobi coincides with Richardson for this choice of ω, this agreement confirms the theoretical result.

Moreover, the contraction factor of Gauss--Seidel is strictly smaller than that of Jacobi for all problem sizes. This explains the faster convergence of Gauss--Seidel and is consistent with the Stein--Rosenberg theorem, which states

$$0 < \rho_{GS} < \rho_J < 1.$$

In addition, the numerical results indicate that the contraction factors satisfy approximately

$$\rho_{GS} \approx \rho_J^2.$$

This observation is in agreement with a classical result due to Young and Frankel, which states that for certain matrices (in particular for consistently ordered matrices such as the discrete 1D Poisson matrix), the spectral radii of the Jacobi and Gauss--Seidel iteration matrices are related by

$$\rho_{GS} = \rho_J^2.$$

This explains quantitatively why Gauss--Seidel converges faster: its asymptotic error reduction per iteration is essentially the square of that of Jacobi.

Finally, for increasing problem size n, both contraction factors approach 1. Hence, the convergence becomes slower for finer discretizations, as the error is reduced less effectively in each iteration.

Exercise 5 (Convergence of Jacobi)

1. Verify that the eigenvalues of A (without prefactor $1/h^2$) are:

$$\lambda_j = 2 - 2\cos\left(\frac{\pi j}{n+1}\right)$$

2. Determine if Jacobi converges and compute the spectral radius $\rho(M^{(J)})$.

1) Eigenvalues and Eigenvectors

Given:

$$\lambda_j = 2 - 2\cos\left(\frac{\pi j}{n+1}\right)$$

$$v^{(j)} = \begin{pmatrix} \sin\left(\frac{\pi j}{n+1}\right) \\ \sin\left(\frac{2\pi j}{n+1}\right) \\ \vdots \\ \sin\left(\frac{n\pi j}{n+1}\right) \end{pmatrix}$$

We show:

$$Tv^{(j)} = \lambda_j v^{(j)}, \quad \text{with} \quad T = h^2 A$$

Component-wise representation

$$(Tv^{(j)})_i = \sum_{k=1}^{n} T_{ik}(v^{(j)})_k$$

Since T is tridiagonal:

$$= T_{i,i-1}(v^{(j)})_{i-1} + T_{i,i}(v^{(j)})_i + T_{i,i+1}(v^{(j)})_{i+1}$$

With:

$$T_{ik} = 2\delta_{ik} - \delta_{i,k-1} - \delta_{i,k+1}$$

we obtain:

$$= -(v^{(j)})_{i-1} + 2(v^{(j)})_i - (v^{(j)})_{i+1}$$

Insert the eigenvector components

$$(v^{(j)})_i = \sin\left(\frac{\pi j i}{n+1}\right)$$

Thus:

$$(Tv^{(j)})_i = 2\sin\left(\frac{\pi j i}{n+1}\right) - \sin\left(\frac{\pi j(i-1)}{n+1}\right) - \sin\left(\frac{\pi j(i+1)}{n+1}\right)$$

Trigonometric identity

$$\sin(a+b) + \sin(a-b) = 2\sin(a)\cos(b)$$

Let:

$$a = \frac{\pi j i}{n+1}, \quad b = \frac{\pi j}{n+1}$$

Then:

$$\sin\left(\frac{\pi j(i+1)}{n+1}\right) + \sin\left(\frac{\pi j(i-1)}{n+1}\right) = 2\sin\left(\frac{\pi j i}{n+1}\right)\cos\left(\frac{\pi j}{n+1}\right)$$

Final result

$$(Tv^{(j)})_i = 2\sin(\dots) - 2\sin(\dots)\cos(\dots)$$

$$= \left(2 - 2\cos\left(\frac{\pi j}{n+1}\right)\right)\sin\left(\frac{\pi j i}{n+1}\right)$$

$$= \lambda_j (v^{(j)})_i$$

Therefore:

$$Tv^{(j)} = \lambda_j v^{(j)}$$

---

2) Convergence of Jacobi

Jacobi converges if:

$$\rho(M_J) < 1$$

Iteration matrix

$$M_J = -D^{-1}(L+U)$$

$$= -D^{-1}(A-D) = I - D^{-1}A$$

Since:

$$D = \frac{2}{h^2}I \quad \Rightarrow \quad D^{-1} = \frac{h^2}{2}I$$

we obtain:

$$M_J = I - \frac{h^2}{2}A = I - \frac{1}{2}T$$

Action on eigenvectors

$$M_J v^{(j)} = \left(I - \frac{1}{2}T\right) v^{(j)}$$

$$= v^{(j)} - \frac{1}{2}Tv^{(j)}$$

$$= v^{(j)} - \frac{1}{2}\lambda_j v^{(j)}$$

$$= \left(1 - \frac{\lambda_j}{2}\right) v^{(j)}$$

Eigenvalues of M_J

$$\mu_j = 1 - \frac{\lambda_j}{2}$$

Substitute λ_j:

$$\mu_j = 1 - \frac{1}{2}\left(2 - 2\cos\left(\frac{\pi j}{n+1}\right)\right)$$

$$= \cos\left(\frac{\pi j}{n+1}\right)$$

---

3) Spectral radius

$$\rho(M_J) = \max_j |\mu_j| = \cos\left(\frac{\pi}{n+1}\right)$$

---

4) Relation to Exercise 4

$$\frac{\|e^{k+1}\|}{\|e^k\|} \longrightarrow \rho(M_J)$$

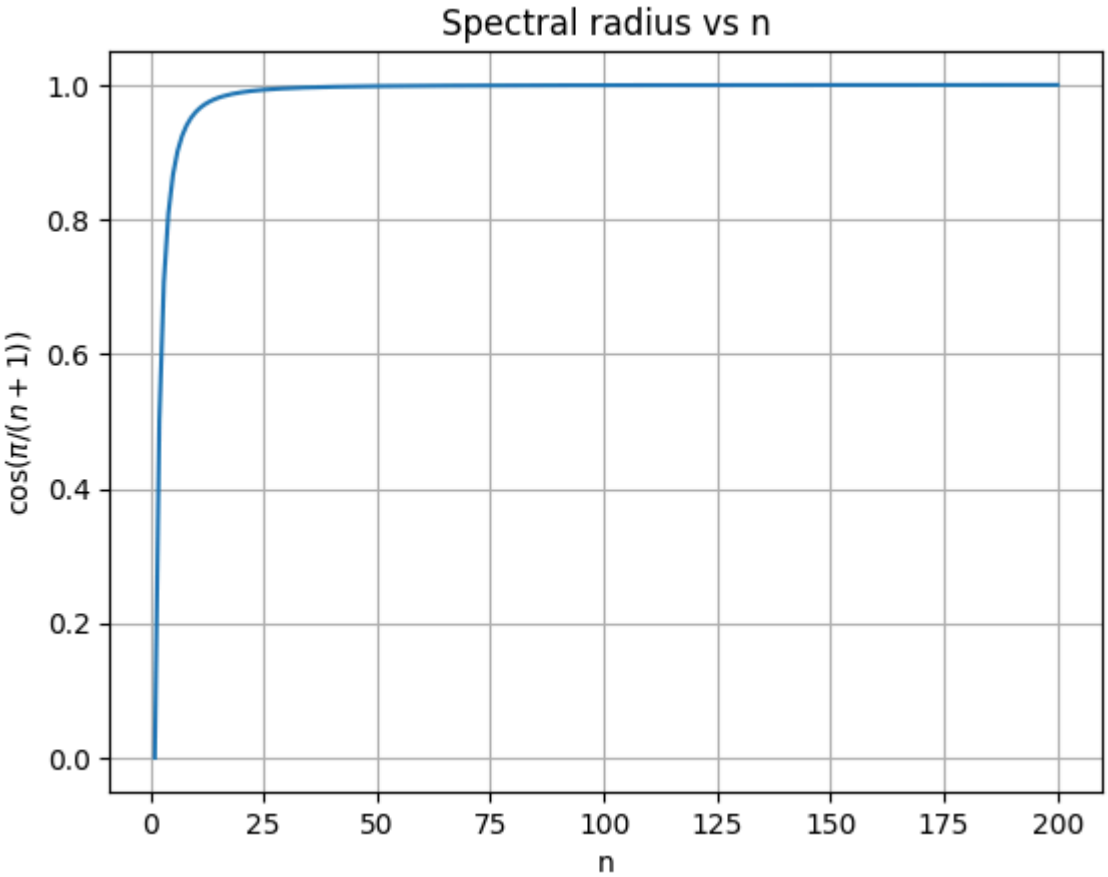
Conclusion

- Jacobi converges since $\rho(M_J) < 1$
- For $n \to \infty$:

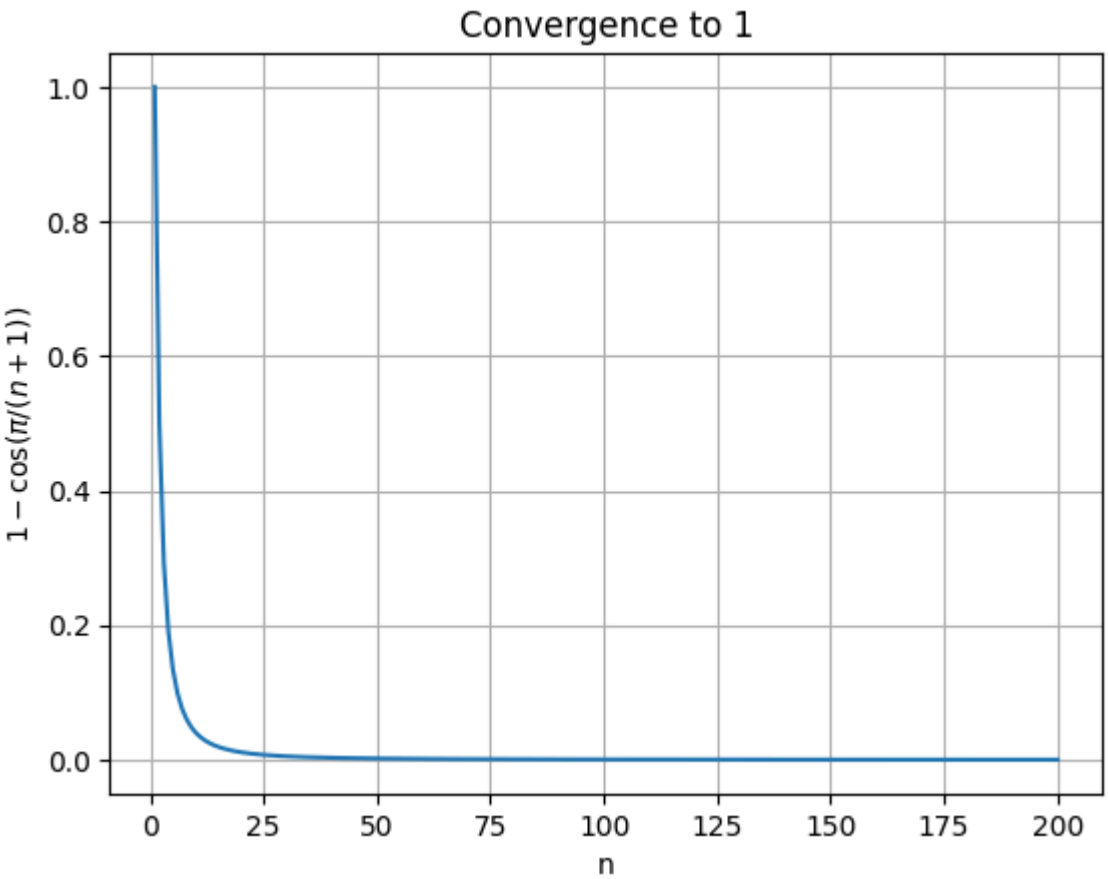
$$\rho(M_J) \to 1$$

→ very slow convergence

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

# range of n
n = np.arange(1, 201)

# compute values
rho = np.cos(np.pi / (n + 1))

# plot
plt.figure()
plt.plot(n, rho)
plt.xlabel("n")
plt.ylabel(r"$\cos(\pi/(n+1))$")
plt.title("Spectral radius vs n")
plt.grid(True)
plt.show()
```



In [2]:
```
plt.figure()
plt.plot(n, 1 - rho)
plt.xlabel("n")
plt.ylabel(r"$1 - \cos(\pi/(n+1))$")
plt.title("Convergence to 1")
plt.grid(True)
plt.show()
```



Exercise 6 (Optimal Damping)

1. Show that for symmetric positive definite matrices, the optimal damping parameter and minimal spectral radius are:

$$\omega_{opt} = \frac{2}{\lambda_{min}(A) + \lambda_{max}(A)}, \quad \rho_{opt} = \frac{\lambda_{max}(A) - \lambda_{min}(A)}{\lambda_{max}(A) + \lambda_{min}(A)}$$

2. Compute these values for the model problem.

1) Richardson Iteration

$$x^{k+1} = x^k + \omega(b - Ax^k) = (I - \omega A)x^k + \omega b$$

Thus, the iteration matrix is:

$$M = I - \omega A$$

Using $T = h^2 A$:

$$M = I - \omega \frac{1}{h^2} T$$

Action on eigenvectors

Let

$$Tv = \lambda v \quad \Rightarrow \quad Av = \frac{\lambda}{h^2} v$$

Then:

$$Mv = (I - \omega A)v = v - \omega Av = v - \omega \frac{\lambda}{h^2} v$$

$$= \left(1 - \omega \frac{\lambda}{h^2}\right) v$$

Thus:

$$\mu_j = 1 - \omega \frac{\lambda_j}{h^2}$$

Spectral radius

$$\rho(M) = \max_j \left| 1 - \omega \frac{\lambda_j}{h^2} \right|$$

---

2) Optimal damping parameter

We want to choose ω such that $\rho(M)$ is minimal.

Define:

$$a = \left| 1 - \omega \frac{\lambda_{\min}}{h^2} \right|, \quad b = \left| 1 - \omega \frac{\lambda_{\max}}{h^2} \right|$$

Goal:

$$\min_\omega \max(a, b)$$

Optimality condition

The optimal convergence rate is achieved when the interval

$$\left[1 - \omega \frac{\lambda_{\max}}{h^2}, \, 1 - \omega \frac{\lambda_{\min}}{h^2} \right]$$

is symmetric with respect to zero.

Thus:

$$\left| 1 - \omega \frac{\lambda_{\min}}{h^2} \right| = \left| 1 - \omega \frac{\lambda_{\max}}{h^2} \right|$$

Resolving the absolute value

We distinguish two cases:

1. $$1 - \omega \frac{\lambda_{\min}}{h^2} = 1 - \omega \frac{\lambda_{\max}}{h^2}$$

which implies $\lambda_{\min} = \lambda_{\max}$, i.e., all eigenvalues are equal.
This is a trivial case and does not occur for the Poisson matrix.

2. Otherwise, there exist $i \neq j$ such that $\lambda_i \neq \lambda_j$.
   In this case, the optimal configuration requires opposite signs, hence

$$1 - \omega \frac{\lambda_{\min}}{h^2} = -\left(1 - \omega \frac{\lambda_{\max}}{h^2}\right).$$

Solve for ω

$$2 = \omega \frac{1}{h^2}(\lambda_{\min} + \lambda_{\max})$$

$$\Rightarrow \omega_{\text{opt}} = \frac{2h^2}{\lambda_{\min}(T) + \lambda_{\max}(T)}$$

Since:

$$\lambda(A) = \frac{1}{h^2}\lambda(T)$$

we obtain:

$$\boxed{\omega_{\text{opt}} = \frac{2}{\lambda_{\min}(A) + \lambda_{\max}(A)}}$$

---

3) Optimal spectral radius

$$\rho_{\text{opt}} = |1 - \omega_{\text{opt}} \lambda_{\min}|$$

Insert ω_{opt}:

$$= \left|1 - \frac{2\lambda_{\min}}{\lambda_{\min} + \lambda_{\max}}\right|$$

$$= \left|\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\min} + \lambda_{\max}}\right|$$

Since everything is positive:

$$\boxed{\rho_{\text{opt}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}}}$$

4) Convergence condition

The Richardson iteration converges if and only if

$$\rho(I - \omega A) < 1.$$

Since the eigenvalues of the iteration matrix $I - \omega A$ are

$$\mu_j = 1 - \omega \lambda_j,$$

this is equivalent to

$$|1 - \omega \lambda_j| < 1 \qquad \forall j.$$

Hence,

$$-1 < 1 - \omega \lambda_j < 1 \qquad \forall j.$$

Right inequality

From

$$1 - \omega \lambda_j < 1$$

we obtain

$$-\omega \lambda_j < 0.$$

Since A is symmetric positive definite, all eigenvalues satisfy

$$\lambda_j > 0 \qquad \forall j.$$

Thus this inequality is automatically satisfied whenever $\omega > 0$.

Left inequality

From

$$-1 < 1 - \omega \lambda_j$$

we obtain

$$\omega \lambda_j < 2,$$

that is,

$$\omega < \frac{2}{\lambda_j} \qquad \forall j.$$

Therefore,

$$0 < \omega < \min_j \frac{2}{\lambda_j}.$$

Since $\lambda_{\max}(A) \geq \lambda_j$ for all j, we have

$$\frac{2}{\lambda_{\max}(A)} \leq \frac{2}{\lambda_j} \qquad \forall j,$$

and hence

$$\min_j \frac{2}{\lambda_j} = \frac{2}{\lambda_{\max}(A)}.$$

So the Richardson iteration converges for

$$0 < \omega < \frac{2}{\lambda_{\max}(A)}.$$

---

5) Application to the Poisson matrix

For the discrete 1D Poisson matrix,

$$\lambda_j(A) = \frac{1}{h^2}\left(2 - 2\cos\left(\frac{j\pi}{n+1}\right)\right).$$

Since $\cos(\theta)$ is decreasing on $[0, \pi]$, it follows that:

- the minimum is attained at $j = 1$,
- the maximum is attained at $j = n$.

Thus,

$$\lambda_{\min}(A) = \lambda_1(A) = \frac{1}{h^2}\left(2 - 2\cos\left(\frac{\pi}{n+1}\right)\right),$$

and

$$\lambda_{\max}(A) = \lambda_n(A) = \frac{1}{h^2}\left(2 - 2\cos\left(\frac{n\pi}{n+1}\right)\right).$$

Now we simplify the cosine term:

$$\frac{n\pi}{n+1} = \frac{(n+1)\pi - \pi}{n+1} = \pi - \frac{\pi}{n+1}.$$

Therefore,

$$\cos\left(\frac{n\pi}{n+1}\right) = \cos\left(\pi - \frac{\pi}{n+1}\right) = -\cos\left(\frac{\pi}{n+1}\right).$$

Hence,

$$\lambda_{\max}(A) = \frac{1}{h^2}\left(2 + 2\cos\left(\frac{\pi}{n+1}\right)\right).$$

Optimal damping parameter

By the general result,

$$\omega_{\text{opt}} = \frac{2}{\lambda_{\min}(A) + \lambda_{\max}(A)}.$$

Substituting the expressions above gives

$$\lambda_{\min}(A) + \lambda_{\max}(A) = \frac{1}{h^2}\left(2 - 2\cos\left(\frac{\pi}{n+1}\right) + 2 + 2\cos\left(\frac{\pi}{n+1}\right)\right).$$

Thus,

$$\lambda_{\min}(A) + \lambda_{\max}(A) = \frac{1}{h^2} \cdot 4 = \frac{4}{h^2}.$$

Therefore,

$$\omega_{\text{opt}} = \frac{2}{4/h^2} = \frac{h^2}{2}.$$

Optimal spectral radius

Again by the general result,

$$\rho_{\text{opt}} = \frac{\lambda_{\max}(A) - \lambda_{\min}(A)}{\lambda_{\max}(A) + \lambda_{\min}(A)}.$$

We compute the numerator:

$$\lambda_{\max}(A) - \lambda_{\min}(A) = \frac{1}{h^2}\left[\left(2 + 2\cos\left(\frac{\pi}{n+1}\right)\right) - \left(2 - 2\cos\left(\frac{\pi}{n+1}\right)\right)\right].$$

Hence,

$$\lambda_{\max}(A) - \lambda_{\min}(A) = \frac{4}{h^2}\cos\left(\frac{\pi}{n+1}\right).$$

Together with

$$\lambda_{\max}(A) + \lambda_{\min}(A) = \frac{4}{h^2},$$

we obtain

$$\rho_{\text{opt}} = \frac{\frac{4}{h^2}\cos\left(\frac{\pi}{n+1}\right)}{\frac{4}{h^2}} = \cos\left(\frac{\pi}{n+1}\right).$$

Final result

Thus, for the Poisson matrix,

$$\boxed{\omega_{\text{opt}} = \frac{h^2}{2}} \qquad \text{and} \qquad \boxed{\rho_{\text{opt}} = \cos\left(\frac{\pi}{n+1}\right)}.$$

Convergence range for the Poisson matrix

From the general convergence condition

$$0 < \omega < \frac{2}{\lambda_{\max}(A)},$$

and using

$$\lambda_{\max}(A) = \frac{4}{h^2},$$

we obtain:

$$0 < \omega < \frac{2}{4/h^2} = \frac{h^2}{2}.$$

Exercise 7 (Smoothing Property of Damped Jacobi)

1. Show that the amplification factor for the j-th error mode is:

$$\mu_j(\omega) = 1 - \omega + \omega \cos\left(\frac{\pi j}{n+1}\right)$$

2. Prove that for $\omega = 2/3$ and high-frequency modes ($j > (n+1)/2$), it holds that $|\mu_j(2/3)| \leq 1/3$.

1) Amplification Factor

The damped Jacobi iteration is defined by

$$x^{k+1} = x^k + \omega D^{-1}(b - Ax^k).$$

Let x be the exact solution of $Ax = b$, and define the error

$$e^k = x - x^k.$$

Then

$$e^{k+1} = x - x^{k+1} = x - \left(x^k + \omega D^{-1}(b - Ax^k)\right).$$

Since $b = Ax$, we obtain

$$e^{k+1} = x - x^k - \omega D^{-1}(Ax - Ax^k) = e^k - \omega D^{-1} A e^k.$$

Hence,

$$e^{k+1} = (I - \omega D^{-1} A) e^k.$$

For the 1D Poisson matrix,

$$A = \frac{1}{h^2} T, \qquad D = \frac{2}{h^2} I,$$

and therefore

$$D^{-1}A = \frac{1}{2}T.$$

Thus, the iteration matrix is

$$M_\omega = I - \frac{\omega}{2}T.$$

Let v_j be an eigenvector of T with eigenvalue

$$\lambda_j(T) = 2 - 2\cos\left(\frac{\pi j}{n+1}\right).$$

Then

$$M_\omega v_j = \left(I - \frac{\omega}{2}T\right)v_j = v_j - \frac{\omega}{2}\lambda_j(T)v_j.$$

Hence the amplification factor is

$$\mu_j(\omega) = 1 - \frac{\omega}{2}\lambda_j(T).$$

Substituting the eigenvalues of T gives

$$\mu_j(\omega) = 1 - \frac{\omega}{2}\left(2 - 2\cos\left(\frac{\pi j}{n+1}\right)\right) = 1 - \omega + \omega\cos\left(\frac{\pi j}{n+1}\right).$$

---

2) High-Frequency Modes

We define the high-frequency modes by

$$j > \frac{n+1}{2}.$$

Then

$$\frac{\pi j}{n+1} > \frac{\pi}{2}.$$

Since also $j \leq n$, we have

$$\frac{\pi j}{n+1} \leq \frac{n\pi}{n+1} < \pi.$$

Therefore,

$$\frac{\pi}{2} < \frac{\pi j}{n+1} < \pi,$$

and thus

$$-1 \leq \cos\left(\frac{\pi j}{n+1}\right) < 0.$$

3) Proof of Smoothing for $\omega = \dfrac{2}{3}$

Substituting $\omega = \frac{2}{3}$ into the amplification factor gives

$$\mu_j\left(\frac{2}{3}\right) = 1 - \frac{2}{3} + \frac{2}{3}\cos\left(\frac{\pi j}{n+1}\right) = \frac{1}{3} + \frac{2}{3}\cos\left(\frac{\pi j}{n+1}\right).$$

For high-frequency modes we have

$$-1 \leq \cos\left(\frac{\pi j}{n+1}\right) < 0.$$

Hence

$$-\frac{1}{3} \leq \frac{1}{3} + \frac{2}{3}\cos\left(\frac{\pi j}{n+1}\right) < \frac{1}{3}.$$

Therefore,

$$\boxed{\left|\mu_j\left(\frac{2}{3}\right)\right| \leq \frac{1}{3}}$$

for all high-frequency modes.

4) Conclusion

This proves the smoothing property of damped Jacobi: high-frequency error components are reduced by at least a factor of $1/3$ in one iteration step. In contrast, low-frequency modes are damped much less strongly. This is exactly why damped Jacobi is an effective smoother in multigrid methods.